

A1

1) ALG_1

Используем структуру, которая умеет проверять, является ли ребро мостом, и удалять ребро

```
struct Edge {
    int id, u, v, w;
};

struct DynamicBridge {
    DynamicBridge(int n, const vector<Edge>& edges);
    bool isBridge(int id);
    void erase(int id);
};

vector<int> ALG1(int n, vector<Edge> e) {
    sort(e.begin(), e.end(),
        [](const Edge& a, const Edge& b) {
            return a.w < b.w;
        });

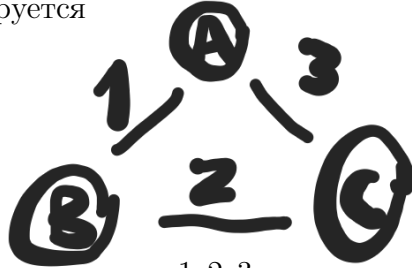
    DynamicBridge db(n, e);
    vector<int> alive(e.size(), 1);

    for (auto &x : e) {
        if (!db.isBridge(x.id)) {
            db.erase(x.id);
            alive[x.id] = 0;
        }
    }

    vector<int> ans;
    for (auto &x : e) {
        if (alive[x.id]) ans.push_back(x.id);
    }
    return ans;
}
```

Сложность $O(m \log m + m \log^2 n)$

MST не гарантируется



Для треугольника с весами 1, 2, 3 удалится ребро веса 1, останутся 2, 3, это не MST

2) ALG_2

Используем DSU. Он хранит компоненты связности и умеет сливать их

```
struct DSU {
    DSU(int n);
    int find(int v);
    bool unite(int a, int b);
};

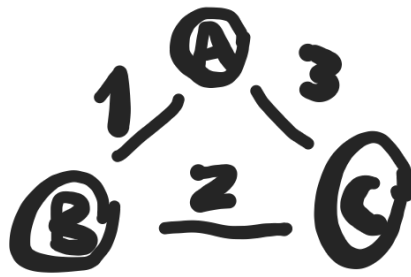
vector<int> ALG2(int n, vector<Edge> e) {
    random_shuffle(e.begin(), e.end());
    DSU dsu(n);
    vector<int> ans;

    for (auto &x : e) {
        if (dsu.unite(x.u, x.v)) {
            ans.push_back(x.id);
        }
    }
    return ans;
}
```

Сложность $O(m\alpha(n))$

Если граф связный, на выходе будет остов

MST не гарантируется



Если сначала взять ребра 2 и 3, получится не MST

3) ALG_3

Используем структуру для леса, которая умеет проверять связность, находить максимальное ребро на пути и удалять его

```
struct DynamicTree {
    bool connected(int u, int v);
    Edge maxEdgeOnPath(int u, int v);
    void link(const Edge& e);
    void cut(int edgeId);
};

vector<int> ALG3(int n, vector<Edge> e) {
    random_shuffle(e.begin(), e.end());
    DynamicTree dt;
    vector<int> in(e.size(), 0);

    for (auto &x : e) {
        if (!dt.connected(x.u, x.v)) {
            dt.link(x);
            in[x.id] = 1;
        } else {
            Edge mx = dt.maxEdgeOnPath(x.u, x.v);
            if (mx.w > x.w) {
                dt.cut(mx.id);
                in[mx.id] = 0;
                dt.link(x);
                in[x.id] = 1;
            }
        }
    }

    vector<int> ans;
    for (auto &x : e) {
        if (in[x.id]) ans.push_back(x.id);
    }
    return ans;
}
```

}

Сложность $O(m \log n)$

Получается MST, потому что в цикле удаляется самое тяжелое ребро

A2

1) DijkstraMULT

Если длина пути считается как произведение, то

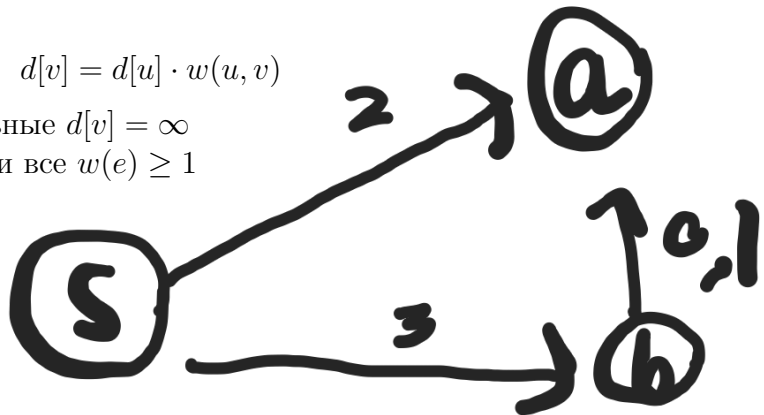
$$d[v] > d[u] \cdot w(u, v)$$

тогда

$$d[v] = d[u] \cdot w(u, v)$$

Старт: $d[s] = 1$, остальные $d[v] = \infty$

Корректно только если все $w(e) \geq 1$



Если есть ребро меньше 1, жадность ломается. Здесь путь через b дает 0.3, а прямой путь дает 2

2) RestoreGraph(dist[[]])

Можно восстановить любой подходящий граф так: для каждой пары $i \neq j$ поставить ребро веса $dist[i][j]$

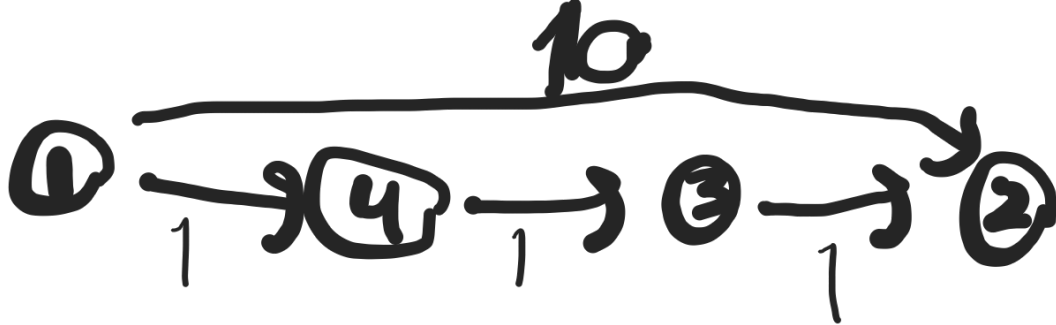
Сложность $O(n^2)$

Именно исходный граф восстановить обычно нельзя

Если нарушено неравенство треугольника или $dist[i][i] \neq 0$, восстановление невозможно

3) Ошибка во Флойде

Ошибка в том, что цикл по k должен быть внешним



Для этого графа правильный ответ из 1 в 2 равен 3, но ошибочный код может оставить 10

4) Одно ребро на кратчайших путях в обе стороны

Да, такой граф возможен



Здесь ребро (u, v) лежит на кратчайшем пути из a в b и из b в a

Новых ограничений нет

При неотрицательных весах подходит Дейкстра

При отрицательных ребрах без отрицательных циклов подходят Беллман - Форд и Флойд - Уоршелл